

## Assignment - 1

1) Explain elements or tokens in Java programs

A token is the smallest individual unit of a Java program that is meaningful to the Java compiler. Every Java program is made up of different types of tokens.

Java tokens are mainly classified into:

1. Keywords
2. Identifiers
3. Literals
4. Operations
5. Separators
6. Comments

1. Key words: Key words are reserved words that have a predefined meaning in Java. They cannot be used as variable, class, or method names.

Ex:- class, public, static, void, int, if, else, for, while, return

Program:-

```
public class Demo {  
    public static void main(String[] args) {  
        int a = 10;  
    }  
}
```

2. Identifiers:- Identifiers are names given to program elements such as classes, methods, variables, and objects.

Ex:- student  
marks  
display  
total

Rules for identifiers:-

1. Must begin with a letter — (or) \$.
2. Cannot begin with a digit.
3. Cannot contain spaces.

4. Cannot be a keyword.

5. Java identifiers are case-sensitive.

Program:-

```
int studentAge = 18;
```

3. Literals:-

Literals are fixed values assigned to variables.

Types include:-

1. Integer literals : 10, 25

2. Floating-point literals : 10.5, 3.14

3. Character literals : "A"

4. String literals : "Java"

5. Boolean literals : true, false

6. Null literal : null

Program:-

```
int age = 18;
```

```
char grade = 'A';
```

```
String name = "Sairam";
```

4. Operators:-

Operators are symbols used to perform operations on data.

Ex:-

Type

Operators

Arithmetic

+, -, \*, /, %

Relational

>, <, >=, <=, ==, !=

Logical

&&

Assignment

=, +=, -=, \*=

Increment / Decrement

++, --

Conditional

?

## 5. Separators:-

Separators are Symbols used to separate different parts of a Java program.

Ex:- ( ), { }, [ ], ; , , , .

Program:-

```
System.out.println("Hello");
```

## 6. Comments:-

Comments are used to provide explanations in the programs. They are ignored by the compiler.

Single-line Comment:-

// This is a comment.

Multi-line Comment:-

/\*

This is a Multi line comment.

\*/

①

②

Describe Command Line arguments.

③

Command line arguments are values passed to a Java program during its execution from the command prompt or terminal.

They are received by the main() method through the String[] args parameter.

Syntax:-

```
public static void main(String[] args)
```

Program:-

```
class CommandLine {
```

```
    public static void main(String[] args) {
```

```
        System.out.println("Name = " + args[0]);
```

```
        System.out.println("Age = " + args[1]);
```

```
    }
```

```
}
```

Output:- Name = Sai ram  
Age = 18

Working:-

If we execute:

java CommandLine Sairam 18

The arguments are stored as:

args[0] = "Sairam"

args[1] = "18"

The index starts from 0(zero).

Important Points:-

1. Command line arguments are passed during program execution.
2. They are stored in the String[] args array.
3. The first argument is stored in args[0].
4. All command line arguments are initially treated as strings.
5. Numeric values can be converted using wrapper methods.

Program:-

```
class Sum {
```

```
    public static void main (String[] args) {
```

```
        int a = Integer.parseInt(args[0]);
```

```
        int b = Integer.parseInt(args[1]);
```

```
        System.out.println("Sum = " + (a+b));
```

```
    }
```

Execution:-

java Sum 10 20

Output:-

Sum = 30

## Advantages:-

- 1) Allows input to be given without using Scanner.
- 2) Useful for passing values while starting a program.
- 3) Useful is automation and Scripting.
- 4) Makes programs flexible.
- \*) Command Line arguments provide a single way to pass input values to a Java program at the time of execution.

## ② Illustrate Data types in Java.

③ A data type specifies the type of data that a variable can store. Java is a strongly typed language, so every variable must have a declared data type.

④ Java data types are classified into:

1. Primitive data types
2. Non-primitive data types

### 1. Primitive Data types:-

Java provides eight primitive data types.

| <u>Data Type</u> | <u>Size</u>   | <u>Example</u>                |
|------------------|---------------|-------------------------------|
| byte             | 8-bit         | byte x = 10;                  |
| Short            | 16-bit        | <del>byte</del> short x = 20; |
| int              | 32-bit        | int x = 100;                  |
| long             | 64-bit        | long x = 1000L;               |
| float            | 32-bit        | float x = 10.5f;              |
| double           | 64-bit        | double x = 20.5;              |
| char             | 16-bit        | char x = 'A';                 |
| boolean          | JVM-dependent | boolean x = true;             |

a) byte:-

Used to store small integer values.

byte age = 18;

b) short:-

Used for integer values larger than byte but smaller than int.

short marks = 500;

c) int:-

The most commonly used integer data type.

int salary = 30,000;

d) long:-

Used for every large integer values.

Long population = 1000000000L;

e) float:-

Used to store decimal values with single precision.

float price = 99.5f;

f) double:-

Used for decimal values with double precision.

double pi = 3.14159;

g) char:-

Used to store a single Unicode Character.

char grade = 'A';

h) boolean:-

Stores either true or false.

boolean result = true;

## 2. Non-Primitive Data types:-

Non-primitive data types are also called reference types.

Examples:-

1) String    2) Array    3) class    4) Interface    5) object.

Program:-

```
String name = "Sairam";
```

```
int[] marks = {80, 90, 85};
```

## Difference Between Primitive and Non-Primitive

### Primitive

- \* provided by Java
- \* Stores actual value
- \* Generally fixed size
- \* Ex:- int, char

### Non-Primitive

- \* Created using classes / interfaces.
- \* Stores reference to an object.
- \* Size can vary.
- \* Ex:- String, arrays.

Demonstrate static with Examples.

The static keyword in Java is used to create members that belongs to the class rather than individual objects.

The Static keyword can be used with:

1. Variables
2. Methods
3. Blocks
4. Nested classes.

## Static Variable:-

A static variable is shared by all objects of a class.

Program:-

```
class student {  
    String name;  
    static String college = "NBKRIST";  
    student (String n) {  
        name = n;  
    }  
    void display() {  
        System.out.println(name + " + college");  
    }  
    public static void main (String[] args) {  
        Student s1 = new Student ("Ravi");  
        Student s2 = new Student ("Sai");  
        s1.display();  
        s2.display();  
    }  
}
```

Output:-

Ravi NBKRIST  
Sai NBKRIST

Here, college is Static, so only one copy is created and shared by both objects.

Static Method:-

A static method belongs to the class and can be called without creating an object.

## Program:-

```
class Demo {  
    static void display () {  
        System.out.println ("Static Method");  
    }  
    public static void main (String [] args)  
    {  
        Demo.display ();  
    }  
}
```

Output:-

Static Method

## Static Block:-

A static block is executed when the class is loaded.

```
class Demo {  
    static {  
        System.out.println ("Static block executed");  
    }  
    public static void main (String [] args) {  
        System.out.println ("Main Method");  
    }  
}
```

Output:-

Static block executed.

Main Method

## Advantages:-

1. Saves memory when a value is common to all objects.
2. Can be accessed using the class name.
3. Static methods can be called without objects.
4. Useful for common constants and utility methods.

Develop a program for Simple calculator.

A calculator program performs arithmetic operations such as addition, subtraction, multiplication, and division.

The Switch statement can be used to select the required operation.

Program:-

```
import java.util.Scanner;
```

```
class Calculator {
```

```
    public static void main (String[] args) {
```

```
        Scanner sc = new Scanner (System.in);
```

```
        System.out.print ("Enter first number: ");
```

```
        double a = sc.nextDouble();
```

```
        System.out.print ("Enter second number: ");
```

```
        double b = sc.nextDouble();
```

```
        System.out.print ("Enter operator (+, -, *, /) : ");
```

```
        char op = sc.next().charAt(0);
```

```
        double result;
```

```
        switch (op) {
```

```
            case '+':
```

```
                result = a + b;
```

```
                break;
```

```
            case '-':
```

```
                result = a - b;
```

```
                break;
```

```
            case '*':
```

```
                result = a * b;
```

```
                break;
```

```
            case '/':
```

```
                if (b != 0)
```

```
                    result = a / b;
```

```
            else {
```

```

System.out.println("Division by zero is not possible");
return;
}
break;
default:
    System.out.println("Invalid Operator");
    return;
}
System.out.println("Result = " + result);
}
}

```

Output:-

Enter first number: 20

Enter second number: 10

Result = 30.0

Prove how Post-Increment / Decrement Differs from Pre-Increment / Decrement.

Java provides increment (++) and Decrement (--) operators.

They can be used in two forms:

1. Pre-Increment / Decrement
2. Post-Increment / Decrement

1. Pre-Increment:-

In pre-Increment, the value is incremented first and then used.

Program:-

```
int a = 5;
```

```
int b = ++a;
```

Execution:-

a = 6

b = 6

## 2. Post-Increment:

In post-Increment, the current value is first, then the value is incremented.

### Program:-

```
int a = 5;  
int b = a++;
```

### Execution:-

b = 5  
a = 6

## 3. Pre-Decrement:

In pre-decrement, the value is decremented first and then used.

### Program:-

```
int a = 5;
```

```
int b = --a;
```

### Execution:-

a = 4

b = 4

## 4. Post-Decrement:

In post-Decrement, the current value is used first and then decremented.

### Program:-

```
int a = 5;
```

```
int b = a--;
```

### Execution:-

a = 4

b = 5

Program :-

class IncrementDecrement {

public static void main (String args []) {

int a = 5;

int b = ++a;

System.out.println ("Pre-Increment:");

System.out.println ("a = " + a);

System.out.println ("b = " + b);

int c = 5;

int d = c++;

System.out.println ("Post-Increment:");

System.out.println ("c = " + c);

System.out.println ("d = " + d);

int e = 5;

int f = --e;

System.out.println ("Pre-Decrement:");

System.out.println ("e = " + e);

System.out.println ("f = " + f);

int g = 5;

int h = g--;

System.out.println ("Post-Decrement:");

System.out.println ("g = " + g);

System.out.println ("h = " + h);

}

}

Output:-

Pre-Increment:

a = 6

b = 6

Post-Increment:

c = 6

d = 5

Pre-Decrement:

e = 4

f = 4

Post-Decrement:

g = 4

h = 5

2 Marks:-

1. Define class

A. A class is a blue print or template used to create objects. It contains data members and methods.

Ex :-

Class Student {

int age;

void display () {

System.out.println(age);

}

}

2. List any four Object-oriented Features.

A. The major OOP Features are:

1. Encapsulation

2. Inheritance

3. Polymorphism

4. Abstraction

3. Develop a Java Program to Add Two numbers and store result in another.

A. Class Add {

public static void main (String [] args) {

int a=10, b=20;

int c = a+b;

System.out.println ("Sum = " + c);

}

}

Output :-

Sum = 30

4. Describe Precedence and Associativity of Operators.

A: Precedence determines which operator is evaluated first.

Associativity determines the direction of evaluation when operators have the same precedence.

Ex:

```
int x = 10 + 5 * 2;
```

\* has higher precedence, so:

$x = 10 + 10 = 20$

5. Use Logical operators solve a suitable problem.

A: Java provides three logical operators:

| <u>Operator</u> | <u>Meaning</u> |
|-----------------|----------------|
| &&              | Logical AND    |
| !               | Logical NOT    |

Example:-

```
if (age >= 18 && age <= 60)
    System.out.println("Eligible");
```

6. How for loop is better than while loop?

A: A for loop is convenient when the number of iterations is known.

```
for (int i=1; i <= 5; i++)
    System.out.println(i);
```

A while loop is generally useful when the number of iterations is not known in advance.

7. Differentiate Break and Continue.

A: break

Continue

1. Terminates the loop completely.

1. Skips the current iteration.

2. Control comes outside the loop.

2. Control goes to the next iteration.

3. Ex: break;  
terminates the loop;

3. Ex: Continue;  
skips the remaining statements of the current iteration.